

Backend Learning - Reference

Bookstore Full-Stack Platform - Tracked Q&A from chat

How APIs and Code Execution Work - High-Level Mental Model

Q: What is an API, at a high level, and how does the client-server / server-DB communication and the code-to-running-program pipeline relate to it?

A: An API is the defined contract for how one system can ask another to do something or return data - not the communication itself. The concept is the same irrespective of language/framework; only the internal implementation of exposing it differs. OpenAPI/Swagger is a machine-readable description of that contract (generated from/alongside code), not the contract itself. Request/response format (HTTP + JSON) is not language-specific - only the code producing/parsing it differs per language. Code execution pipeline: write code (language-specific syntax) -> language-specific tooling converts it (compiler/interpreter/JIT - the editor is not part of this conversion) -> the machine executes the result. .NET specifics: C# compiles to IL, then the CLR's JIT compiles IL to machine code at runtime - this is why compiled .NET output is portable across OSes.

Compile Time vs Runtime

Q: In generic terms, irrespective of language and framework, how would you explain compile time and runtime?

A: Compile time = the phase before the program runs: syntax validation, type checking, reference/name resolution, translation to a lower-level form (machine code/bytecode/IL) - errors caught here stop the build before anything executes. Runtime = the phase where the translated program actually executes: reading input, running logic, producing output, handling requests, rendering UI - errors here only surface when that specific code path is actually reached. How much happens at each phase varies by language: fully compiled (C, Rust) does almost everything at compile time; fully interpreted (plain Python, older JS) barely separates the phases; hybrid (.NET, Java) compiles to an intermediate form (IL/bytecode) then JIT-compiles to machine code during runtime. The concept - 'checked/prepared beforehand' vs 'actually happening now' - is universal; only where the line falls and how much is caught early differs per language.

High-Level vs Low-Level Language

Q: Is high-level just 'code understandable by humans' and low-level just 'machine code understandable by the machine but not humans'?

A: The distinction is abstraction from hardware, not 'human-written vs machine-written' or readability. High-level (C#, Python, Java) abstracts away memory addresses/registers/CPU instructions; low-level (machine code, assembly) is close to or directly operates on hardware. Assembly language is the key counter-example to the readability framing: it's low-level (hardware-specific, one instruction = one CPU operation) but is human-written, using mnemonics (MOV, ADD, JMP) instead of raw binary - proving low-level does not mean unreadable by humans. Spectrum: machine code (lowest, binary) -> assembly (low, human-readable mnemonics) -> C#/Java/Python (high, hardware-abstracted). Level = distance from hardware; readability tracks it loosely but isn't the definition.

.NET SDK Components

Q: What exactly are the SDK, CSC, CLI, project templates, build tools, CLR, and BCL, and how do they relate to each other?

A: SDK = complete local toolchain to build/run .NET apps: CSC (C# compiler, compiles C# to IL), CLI (build/create project etc.), project templates (predefined project code), build tools (compile/link/package), plus a bundled .NET Runtime. Runtime (bundled inside the SDK, also independently installable) = CLR (takes IL, JIT-compiles to machine code, executes it) + BCL (library of prebuilt/precompiled classes and methods used by the application). Correction: CLR/BCL are part of the Runtime, not directly 'a subset of SDK' - the SDK contains the Runtime, not the reverse. Production servers often install Runtime-only, since they don't need the compiler/CLI/build tools to just run an already-built app.

CLR Internals (JS Engine Analogy)

Q: Is CLR like the JS engine, with its own internal components specific to running .NET, similar to how the JS engine has execution contexts, call stack, heap, callback queue, event loop?

A: Yes. CLR components: JIT Compiler (IL to machine code at runtime), Managed Heap + generational Garbage Collector, native OS-level thread management (real multithreading, unlike JS's single-thread plus event loop), Type Safety/Verification of IL before execution, structured exception handling (try/catch/finally), and legacy Code Access Security (mostly deprecated). Key difference: JS's call stack/callback queue/event loop exist to fake concurrency on one thread; CLR doesn't need that pattern since it does concurrency via real OS threads (Task/async-await use a thread pool, not a single-thread event loop).

Thread / Multithreading / Concurrency

Q: What exactly is a thread? What does '.NET is multithreaded' mean? What is running code concurrently?

A: A thread is a logical construct (not a physical CPU component) representing one independent stream of execution - concretely, an instruction pointer (which line of machine code runs next) plus its own call stack (local variables, call history) plus its own register state. A process can have one or many threads, each an independent path of execution through the code, sharing the process's memory/data. The OS schedules threads, not whole programs, on a CPU core - the thread's instruction-pointer-plus-stack state is what gets loaded onto a core, run for a time slice, then swapped out (or run simultaneously on a different core, on multi-core). The CPU core is the physical thing doing work; the thread is the record of whose instructions it's currently executing. '.NET is multithreaded' means a .NET app can create and run multiple threads simultaneously, genuinely scheduled onto separate CPU cores for true parallel execution - unlike JS, which runs on a single thread and fakes concurrency via the event loop. Concurrency vs parallelism: Concurrency = multiple tasks making progress over the same time window, possibly interleaved on one core (what JS's event loop and single-core machines achieve). Parallelism = multiple tasks literally executing at the same instant on separate CPU cores (requires multiple threads plus multiple cores). .NET supports both: Task/async-await typically gives concurrency (freeing a thread during I/O wait); Parallel.For or explicit multiple threads/tasks for CPU-bound work gives true parallelism.

Project Types and Build Output

Q: A Project is a single compilable unit producing one output (.dll or executable). Is the compiled IL file that output? What are the different project types in .NET and their outputs?

A: Correction: IL isn't a separate standalone file; it's packaged inside the assembly (.dll/.exe) itself, wrapped with metadata (version, type definitions, manifest). No intermediate .il file exists in a normal build (only via explicit disassembly with ildasm). Project types and outputs: Class Library (Microsoft.NET.Sdk) -> .dll only, no entry point. Console App (Microsoft.NET.Sdk) -> .dll (IL) + native apphost (.exe on Windows) that launches the runtime and loads the dll. ASP.NET Core Web App/API (Microsoft.NET.Sdk.Web) -> same, self-hosts via Kestrel. Worker Service (Microsoft.NET.Sdk.Worker) -> .dll + apphost exe. WPF/WinForms (Microsoft.NET.Sdk + UI flags) -> .dll + apphost exe, Windows-only. Test project (xUnit/NUnit/MSTest) -> .dll, run via a test host process. Blazor WebAssembly (Microsoft.NET.Sdk.BlazorWebAssembly) -> .dlls compiled to WASM, served as static files. Apphost nuance: the native exe is just a launcher stub starting the CLR and handing it the IL dll - actual logic always lives in the .dll (IL form) regardless of project type. Exception: Native AOT publish produces a genuine native machine-code executable directly - no IL, no CLR/JIT needed at runtime.

Project Files, Assembly, Build vs Run

Q: Is a Project basically a .csproj file? What are the different file extensions created in .NET? What exactly is an assembly? What happens when we Build vs Run a project or a solution?

A: Project = .csproj (XML) - defines target framework, SDK type, package references, included source files, build settings; it's the project's identity/config, not the code itself. File extensions: .cs (source code), .csproj (project file), .sln (solution file, groups multiple projects, text-based not XML), .dll (compiled assembly), .exe (native apphost launcher, Windows), .pdb (debug symbols mapping IL back to source), .deps.json (dependency list), .runtimeconfig.json (runtime version/settings needed), .nupkg (NuGet package), .resx (resource files embedded into assembly), .editorconfig (code style rules, not compiled). Assembly = the actual unit of deployment/versioning in .NET: a .dll or .exe containing compiled IL plus a manifest (metadata: assembly name, version, referenced assemblies, exported types) - the thing the CLR actually loads and executes. One project typically produces exactly one assembly. Build (project): CSC compiles .cs files into IL, packaged into the assembly, written to bin/Debug/net10.0/ (or Release) along with .pdb, .deps.json, .runtimeconfig.json, and the apphost .exe; nothing executes; NuGet dependencies are restored/copied into the output folder. Build (solution): builds every project in the .sln, respecting project-reference dependency order. Run (project): builds first if changed, then the apphost .exe launches, starts the CLR, the CLR loads the assembly's IL, JIT-compiles it to machine code, and executes Main. Run (solution): only relevant if the solution has one designated startup project - Run executes that project's output; other projects are built as dependencies, not executed themselves.

Assembly Clarification

Q: Is 'Assembly' just a name for the output generated after CSC's compilation step - the same mechanism for all project types, only the content/file type generated differs based on project type?

A: Correct on the concept, with one precision: assembly isn't just a name - it's a specific packaging format (IL plus a manifest with metadata: version, referenced assemblies, exported types). CSC produces that structure for every project type. The mechanism is identical everywhere (CSC compiles C# to IL, packages it as an assembly); what differs per project type is only the output extension and how it's hosted afterward - Class Library gives a .dll assembly with no host/entry point; Console/Web/Worker/Desktop apps give a .dll assembly plus a native apphost .exe wrapper that launches it; Blazor WASM compiles assemblies further to WASM served as static files. Same compilation mechanism and assembly format as the core output - project type only decides what gets built around that assembly.

Live Demo of Assembly Creation

Q: To see this assembly creation live in action, do I need to create a project in dotnet and run it?

A: Yes. Steps: 1) `dotnet new console -n TestApp` creates the project (.csproj, Program.cs). 2) `dotnet build` invokes CSC and produces the assembly - check `TestApp/bin/Debug/net10.0/` for `TestApp.dll` (the assembly), `TestApp.exe` (apphost, Windows), `TestApp.pdb`, `.deps.json`, `.runtimeconfig.json`. 3) `dotnet run` builds (if needed) then executes, showing `Console.WriteLine` output. To confirm IL is packaged inside the .dll, disassemble it with `ildasm TestApp.dll` (Windows, ships with Visual Studio) or `dotnet-ildasm /ILSpy` (cross-platform) - shows the raw IL instructions and manifest metadata inside the assembly.

`-n` Flag in `dotnet new`

Q: What is -n in the command `dotnet new console -n TestApp` ?

A: `-n` is short for `--name` - it sets the name of the project (and the folder it creates). `dotnet new console -n TestApp` creates a folder `TestApp/` with `TestApp.csproj` and `Program.cs` inside, and the project/assembly itself is named `TestApp`. Without `-n`, it uses the current directory's folder name as the project name instead.

`obj` Folder Contents

Q: Running `dotnet new console` created `TestApp.csproj` and `Program.cs`, plus an `obj` folder with other files inside - what exactly are these files, and why has the `obj` folder been created?

A: `obj/` holds intermediate build artifacts - working files MSBuild/CSC generate during compilation, not meant to be shipped or committed to git. Typical contents: `project.assets.json` (the resolved NuGet dependency graph after restore), `TestApp.csproj.nuget.g.props` / `.g.targets` (auto-generated MSBuild files wiring in NuGet package references), `TestApp.GlobalUsings.g.cs` (auto-generated implicit global usings, since `ImplicitUsings` is enabled by default), and a `Debug/net10.0/` subfolder with intermediate compiled output (a pre-final copy of `TestApp.dll`, `TestApp.pdb`, `.deps.json`, `.runtimeconfig.json`, and a `ref/` folder used for faster incremental builds). `obj/` is scratch space for the build process itself (restore metadata, incremental-build caching, generated code); `bin/` is the actual deployable output copied over once build succeeds - this split lets MSBuild do incremental builds efficiently. Both `obj/` and `bin/` should always be in `.gitignore`, since they're regenerable from source.

Program.cs, .csproj, and What a Console App Is

Q: What are Program.cs and TestApp.csproj? Is this the same set of files created for every new console app? So what exactly is a console app?

A: Program.cs is the actual C# source file, entry point of the app - in modern .NET (top-level statements, default since .NET 6+), it's just executable code directly in the file, no explicit class Program or static void Main wrapper needed since the compiler generates that scaffolding. TestApp.csproj is the project file (MSBuild XML: target framework, SDK type, implicit usings, nullable settings, package references). Yes - every `dotnet new console` scaffolds the same two files and structure, regardless of project name; that's what a template does. A console app is a project type whose SDK output is a .dll plus an apphost .exe designed to run in a terminal/command-line environment - no UI framework, no web server, no windowing system. It reads input (args, stdin) and writes output (stdout/stderr) as text, and has a defined entry point (Main, or top-level statements standing in for it) where execution starts when the apphost launches it. It's the minimal, UI-less, host-less project type - everything else (Web API, WPF, Worker) is essentially 'console app plus additional hosting/framework layered on top.'

Which Files to Open in VS 2026

Q: I'll be editing both files but writing code in Program.cs - which files should I open in the IDE, which is VS 2026?

A: Open the folder (or generate a .sln via `dotnet new sln` plus `dotnet sln add TestApp.csproj`) in VS - not individual files directly. Program.cs is where code actually gets written day to day. TestApp.csproj isn't hand-edited constantly - VS exposes most of it through UI (Project > Properties, NuGet Package Manager); open it directly only for things the UI doesn't expose cleanly, such as manually adding an MSBuild property. Practical flow: File > Open > Folder pointed at TestApp/ - VS 2022+ supports folder-based opening without needing a .sln first; VS auto-detects the .csproj and loads it into Solution Explorer, with Program.cs ready to edit.

Why a Solution Is Needed

Q: A solution just groups related project files into one unit for navigation in the IDE - if folder-based opening is supported, is a .sln even needed? What exactly is the need for a Solution?

A: Correct on what a solution is, but that's only half of it - the real need shows up once there's more than one project. For a single project, folder-based opening is genuinely enough, no .sln needed. The .sln earns its place with multiple related projects that need to be treated as one unit for building, not just browsing: build orchestration (building the .sln builds every project in the correct dependency order); one designated startup project (tells the IDE which project's output actually runs); shared build configuration (one Debug/Release switch applies across all projects at once); and cross-project references tracked at the solution level (IntelliSense, go-to-definition across projects, build ordering). Concretely: a lone console app doesn't need a .sln, but a Clean Architecture split into several separate, referencing projects is exactly the scenario a .sln exists for.

Which File/Command to Run in VS

Q: Every file I click on in VS makes the Run button select that file - so which file should I run? Or should I run `dotnet build` for the whole folder? Also, what command should I run in the terminal to see the output?

A: Don't run individual files - run the project (TestApp.csproj), which VS should auto-detect as the startup item in Open Folder mode. The Run-button relabeling to whatever file is selected usually means VS's Startup Item dropdown isn't locked to the project - check the dropdown next to Run and explicitly select the project if needed. Simpler and unambiguous: use the terminal, `dotnet run`, from inside TestApp/ - builds (if needed) then executes Program.cs's entry point regardless of which file is open/selected. `dotnet build` alone only compiles, it doesn't execute. `dotnet run` builds the project (if anything changed) and then executes it, so Console.WriteLine output shows in the terminal.

The `bin` Folder

Q: After running build, a bin folder was created - is the assembly the .dll, with .exe also created? Will bin get replaced/recreated every build/run, and why does a bin folder get created - what is its need?

A: Correct - .dll is the assembly, .exe is the apphost wrapper. bin/ gets regenerated on every build/run - MSBuild recompiles what changed and overwrites the output files in place. bin/ exists as the final, deployable output location (short for 'binaries'), separate from obj/: obj/ is build-process scratch space (intermediate/incremental-build artifacts); bin/ is the actual finished output to copy elsewhere to deploy/run, what `dotnet publish` further packages for deployment. Structure: bin/Debug/net10.0/ (or Release/net10.0/ depending on build configuration) - the Debug/Release subfolder lets both configs build without overwriting each other; net10.0 names the target framework. Like obj/, it's regenerable from source and belongs in .gitignore.

Where to Run `ildasm`

Q: Where should I run `ildasm TestApp.dll` to see the IL file content?

A: Run it from the Developer Command Prompt for VS 2026, not a regular terminal - that's where ildasm.exe is on PATH. Navigate to the actual .dll location (`cd TestApp\bin\Debug\net10.0`) then run `ildasm TestApp.dll` - it opens ildasm's GUI showing the assembly's namespace/class tree; expand to the method (e.g. Main) and double-click to see the actual IL instructions. Caveat: ildasm isn't guaranteed installed by default with VS 2026 - it ships via .NET SDK workload tools or needs adding via Visual Studio Installer > Individual Components > 'MSIL Disassembler'. If not found on PATH, a cross-platform alternative as a global dotnet tool works from any regular terminal: `dotnet tool install -g ilspycmd` then `ilspycmd TestApp.dll -il` - prints the IL directly to the terminal, no GUI needed.

Where to Run `ilspycmd`

Q: Should `dotnet tool install -g ilspycmd` and `ilspycmd TestApp.dll -il` be run inside the project folder or can they be run anywhere?

A: `dotnet tool install -g ilspycmd` can be run anywhere - it's a global tool, installs once to the user profile, and isn't tied to any project. `ilspycmd TestApp.dll -il` needs to run from wherever the .dll actually is, since it references the file by relative path - either `cd` into `TestApp\bin\Debug\net10.0` first and run it as-is, or run it from anywhere by giving the full/relative path to the dll: `ilspycmd path\to\TestApp.dll -il` .

Simple bin/obj Recap + CLI vs VS Project Creation

Q: Give simple explanations for the bin and obj folders. Also, a project can be created using dotnet commands from the terminal or using File > New Project/Solution inside VS - both return the same result, right?

A: obj is scratch space used during the build - it holds intermediate files (dependency info, incremental-build cache, generated code) the compiler needs while working, not the final output. bin is the actual finished output - it holds the compiled assembly (.dll), the launcher (.exe), debug symbols, and config files, i.e. what you'd actually run or deploy. Both regenerate every build and are gitignored. Yes, same result - `dotnet new console -n TestApp` and VS's File > New > Project (Console App template) both scaffold the identical structure: TestApp.csproj, Program.cs, same default settings. VS's dialog is just a GUI wrapper around the same underlying `dotnet new` template engine - no difference in what gets generated.